

Beyond LLMs: A Neuro-Symbolic Framework for Commonsense Reasoning in Robotics

Cono Cirone

January 2026

1 Problem Statement and Motivation

Commonsense knowledge (CSK) comprises the largely implicit regularities humans rely on in everyday activities (e.g., that knives are typically sharp and used for cutting). In robotics, insufficient CSK often manifests as brittle behavior when a system encounters underspecified instructions, unknown objects, or unexpected environments. Recent work has evaluated large language models (LLMs) on language-driven benchmarks such as RoboCSKBench [5], demonstrating their broad coverage and robust handling of linguistic variation. However, their outputs may be stochastic, difficult to calibrate, and nontrivial to verify. Furthermore, current LLMs exhibit limited explicit reasoning capabilities and frequently generate unfaithful or incorrect answers.

Historically, symbolic AI approaches have been utilized to support traceable inferences, systematic debugging, and explicit constraint checking. Yet, purely symbolic methods risk coverage gaps and brittleness, and they require substantial knowledge-engineering effort to manually craft task-specific background knowledge.

This thesis proposes a hybrid neuro-symbolic framework that circumvents the limitations of both pure LLM and pure symbolic approaches. By leveraging LLMs as dynamic semantic parsers and coupling them with robust logic solvers (e.g., Answer Set Programming) and Inductive Logic Programming (ILP), the system can automate the translation of natural language into structured, machine-interpretable logic. Evaluated against RoboCSKBench, this approach aims to deliver scalable, verifiable commonsense reasoning while entirely bypassing the manual rule-authoring bottleneck.

2 Literature Review

The integration of Commonsense Knowledge (CSK) into cognitive robotics requires robust frameworks capable of bridging high-level semantic reasoning with low-level perception. As highlighted in recent systematic reviews, this integration typically relies on specialized knowledge processing infrastructures, symbolic logic solvers, and probabilistic extensions to handle open-world uncertainty [6].

2.1 RoboCSKBench Benchmark

To systematically evaluate how well an agent can leverage commonsense knowledge in household robotics, this thesis builds on **RoboCSKBench**, a language-driven, multi-task benchmark designed to assess *embodied* commonsense capabilities in dynamic domestic environments [5]. The benchmark reformulates heterogeneous knowledge sources into standardized NLP-style tasks. Concretely, it comprises five household-relevant tasks:

- **Tidy Up:** Given a household object, the model is expected to return prototypical locations where the object is likely to be found.
- **Tool Usage:** Focuses on affordances, evaluating whether a model can connect objects to plausible uses.
- **Meta-Reasoning:** Evaluates whether a given robotic hardware setup can execute a natural-language instruction based on physical constraints.
- **Table Setting:** Given a meal, the model must select appropriate cutlery and plate type, reflecting object-role knowledge.
- **Procedural Knowledge:** Tests temporal ordering by asking the model to deduce which recipe step occurs before another based on causal dependencies.

2.2 Knowledge Processing and Symbolic Logic

To execute complex tasks, robots require architectures that can organize and query heterogeneous knowledge. Ontologies like the **Socio-physical Model of Activities (SOMA)** [1] provide the necessary axioms to reason about object roles and environment-specific constraints. Symbolic approaches provide the rigorous formalisms necessary for maintaining consistency in robot planning. **Answer Set Programming (ASP)** has proven particularly effective for housekeeping scenarios [3], enabling non-monotonic reasoning to handle defaults and exceptions.

2.3 Neuro-Symbolic AI and Rule Induction

Recent neuro-symbolic approaches integrate LLMs into formal reasoning frameworks, exploiting their effectiveness in translating natural language into structured representations while avoiding the knowledge engineering bottleneck. Frameworks like LLM2LAS [2] demonstrate that LLMs can act as few-shot semantic parsers to extract structured features from text, which Inductive Logic Programming (ILASP) then transforms into interpretable logic rules. This combination retains the semantic breadth of LLMs while ensuring deductive validity.

2.4 Reinforcement Learning for Logic Generation

While offline rule induction translates static facts, evaluating complex ad-hoc scenarios requires dynamic logic formulation. Recent research highlights Reinforcement Learning (RL) as a valuable strategy for refining LLM-based logic generation. Techniques like Group Relative Policy Optimization (GRPO) [4] have been successfully applied to optimize the generation of executable logic programs (e.g., Prolog) by using execution-based rewards to guide training. This enables models to produce logically consistent and syntactically accurate code by directly integrating reasoning-driven feedback.

3 Planned Methodology: Neuro-Symbolic Approach

To address the stochastic limitations of LLMs and the knowledge-engineering bottleneck of classical AI, I propose a **Neuro-Symbolic Framework**.

3.1 Module 1: Property-Based Semantic Parsing and Rule Induction

For tasks requiring broad coverage of object associations and affordances, specifically *Tidy Up* and *Tool Usage*, the system employs a neuro-symbolic pipeline that maps unstructured text to formal properties. To strictly prevent data leakage, this module separates the learning of generalized commonsense rules from the live evaluation on the RoboCSKBench dataset.

Phase 1: Offline Knowledge Extraction and Rule Induction

Prior to evaluating the system on RoboCSKBench, the framework establishes its background knowledge and reasoning rules using a large-scale semantic knowledge graph (e.g., ConceptNet), replacing narrower, highly noisy datasets. To prevent semantic bloat, a subgraph extraction step first filters the graph for nodes strictly relevant to domestic environments.

- **Feature Extraction (LLM as Semantic Parser):** Constrained by the rigorous axioms of the Socio-physical Model of Activities (SOMA) ontology, the LLM processes the raw relationships from the filtered ConceptNet subgraph. It acts strictly as a parser rather than a generator, mapping human-centric semantic associations exclusively into formal, physically grounded properties.
 - *Tidy Up Example:* Filtering ConceptNet’s noisy edges for “milk” to extract the SOMA-compliant `has_property(milk, requires_cooling)`.
 - *Tool Usage Example:* Extracting `has_property(frying_pan, heavy)` and `has_property(frying_pan, rigid)`.
- **Rule Induction (ILASP):** These strictly SOMA-compliant properties serve as the formal Background Knowledge. By pairing these properties with known object-location or object-task relationships from ConceptNet (e.g., `AtLocation` or `UsedFor` edges) serving as training examples, the ILASP system autonomously induces generalizable Answer Set Programming (ASP) rules. These learned rules are deductive, “frozen,” and loaded into the ASP solver to ensure verifiable execution.
 - *Tidy Up Rule:* `location(X, fridge) :- has_property(X, requires_cooling)`.
 - *Tool Usage Rule:*
`can_substitute(X, hammer) :- has_property(X, heavy), has_property(X, rigid)`.

Phase 2: Online Inference and Label Alignment (Zero-Shot Evaluation)

During the live benchmark evaluation, the framework processes unseen natural language queries dynamically, ensuring a true zero-shot test of the system’s reasoning capabilities.

- **Dynamic Semantic Parsing (LLM):** Given a target query, the LLM acts purely as a semantic parser. It does not attempt to guess the answer; it solely extracts the formal properties of the objects mentioned in the multiple-choice options using the predefined ontology.
 - *Tool Usage Application:* If the benchmark asks to find a hammer substitute among a sponge, banana, and frying pan, the LLM parses the candidates into `has_property(sponge, soft)`, `has_property(frying_pan, heavy)`, etc.
- **Symbolic Constraint Solving (ASP Solver):** The ASP solver applies the frozen ILASP rules to the freshly extracted properties to mathematically deduce valid task states.
 - *Tool Usage Application:* The solver evaluates the candidates against the frozen `can_substitute(X, hammer)` rule, formally rejecting the sponge and deducing the frying pan as the only logically valid substitute.
- **Output Projection (LLM):** To satisfy the benchmark’s specific evaluation metrics, a lightweight LLM call semantically projects the solver’s localized output back into natural language to match the standardized multiple-choice label space of RoboCSKBench.

3.2 Module 2: Dynamic Logic Synthesis and Constraint Solving

The remaining tasks in the benchmark, **Meta-Reasoning**, **Table Setting**, and **Procedural Knowledge**, require the evaluation of ad-hoc hardware constraints, compositional matching, and temporal sequencing. Because pre-computing static rules for every possible sequence or hardware permutation is intractable, this module shifts from offline rule induction to *Dynamic Logic Synthesis*.

Phase 1: Offline GRPO Fine-Tuning

To overcome the lack of human-annotated datasets mapping natural language constraints to formal logic, the framework utilizes a procedurally generated proxy dataset of robotic scenarios. An open-weight LLM (e.g., Qwen2.5-Coder) will undergo domain-specific fine-tuning using Group Relative Policy Optimization (GRPO).

This optimization relies strictly on execution-based rewards: the model generates candidate Prolog programs that are evaluated in a background symbolic engine via a Python-Prolog interface (e.g., PySwip). The model learns to internalize formal logic entirely through trial and error by maximizing a structured reward function. Specifically, the model receives a full correctness reward (+1.0) if the Prolog interpreter’s output matches the expected answer, a format reward (+0.5) for strict template adherence, and penalties for incorrect results (−1.0) or syntax errors (−0.5) that prevent execution.

Phase 2: Online Dynamic Translation and Inference

During zero-shot evaluation, the fine-tuned LLM translates the novel natural language scenario directly into an executable logic program. This generated code is piped into the symbolic engine, which deterministically solves the query and prevents probabilistic hallucinations.

Operational Examples by Task Category:

- **Meta-Reasoning (Constraint Satisfaction):**

The LLM translates the hardware specifications into Prolog facts and synthesizes the task constraints based on parametric commonsense.

```
% Synthesized Rules & Facts
mobility(mobile, configC).
arms(1, configC).
suitable(fetch_apple, Config) :-
    mobility(mobile, Config),
    arms(N, Config), N >= 1.
```

The Prolog engine evaluates `?- suitable(fetch_apple, configC).` and deterministically outputs `True`.

- **Table Setting (Compositional Matching):**

The LLM ingests the benchmark’s allowed label space as valid variables and extracts the physical properties of the target meal.

```
% Synthesized Rules & Facts
available_cutlery(spoon).
meal.loaded_potato_soup.
property.loaded_potato_soup, is_liquid.

suitable_cutlery(Meal, Cutlery) :-
    property(Meal, is_liquid),
```

```
available_cutlery(Cutlery),
Cutlery = spoon.
```

The engine executes `?- suitable_cutlery(loaded_potato_soup, C). yielding C = spoon.`

- **Procedural Knowledge (Temporal Sequencing):**

To prevent LLM hallucination based on mere textual proximity, the framework maps procedures to classical AI planning logic (Preconditions and Effects). The LLM extracts state transitions, while the Prolog engine utilizes a universal temporal logic rule to verify sequences.

```
% Synthesized Facts & Universal Rule
produces(action_cook, state_cooked_pasta).
requires(action_bake, state_cooked_pasta).

is_before(ActionA, ActionB) :-
    produces(ActionA, State),
    requires(ActionB, State).
```

If queried with `?- is_before(action_bake, action_cook).`, the solver finds no causal link where baking produces a state that cooking requires, deterministically returning `False` (which is mapped to "No").

4 Experimental Evaluation

The proposed neuro-symbolic framework will be evaluated across the five core tasks of the RoboCSKBench dataset. To rigorously assess the viability of this hybrid architecture, I will report both end-to-end task performance and intermediate logic execution metrics, effectively isolating the performance of the neural translation layer from the symbolic solver.

- **End-to-End Task Metrics:** Following the standard RoboCSKBench evaluation protocol, I will report Accuracy, Precision, Recall, and F1-score for binary and multiple-choice tasks (e.g., *Meta-Reasoning*, *Procedural Knowledge*, *Tool Usage*). For generative tasks (e.g., *Tidy Up*), ranking metrics such as Mean Reciprocal Rank (MRR) will be utilized to evaluate the relevance of the solver’s outputs.
- **Logic Execution Metrics ($Pass@k$ and $Pass^k$):** To quantitatively evaluate the success of the GRPO fine-tuning in Module 2, I will track the model’s ability to reliably synthesize functionally correct logic. Using the $Pass@k$ metric, a task is considered successfully solved if at least one of k generated Prolog candidate programs executes and yields the correct expected answer. Additionally, I will report the stricter $Pass^k$ metric, which requires all k generated candidates to execute correctly, providing a rigorous measure of the model’s logical consistency and resistance to hallucination.
- **Ablation Studies & Error Analysis:** Because neuro-symbolic pipelines are susceptible to compounding errors, each query will be processed through a normalization layer prior to reasoning. I will conduct ablation studies using an oracle normalization setup to cleanly disentangle LLM semantic parsing failures from downstream symbolic reasoning failures. Finally, a structured error analysis will categorize residual failures into specific taxonomy classes: lexical grounding errors, ontology mismatches, syntax compilation failures, and missing solver constraints, providing representative examples for each category to guide future work.

5 Project Timeline

The project will be carried out over a period of six months (24 weeks). The final four weeks are strictly reserved for drafting and finalizing the thesis document. The bi-weekly schedule is detailed in Table 1.

Weeks	Phase	Key Goals & Milestones
1–2	Familiarization	Become familiar with both approaches; understand their respective problems and limitations.
3–5	Module 1 (Extraction)	Extract domestic subgraphs from ConceptNet; implement LLM as semantic parser using SOMA ontology.
6–8	Module 1 (Induction)	Use ILASP for rule induction; load frozen ASP rules to evaluate Tidy Up and Tool Usage tasks.
9–11	Module 2 (Prep)	Shift to Dynamic Logic Synthesis; construct procedurally generated proxy dataset for logic translation.
12–15	Module 2 (Fine-tuning)	Execute GRPO fine-tuning of the chosen LLM using PySwip execution rewards; test dynamic translation.
16–18	Evaluation	Run end-to-end evaluation across all tasks; compute Accuracy, MRR, Pass@k, and Pass^k metrics.
19–20	Analysis	Conduct oracle ablation studies; perform structured error analysis (lexical, ontology, syntax failures).
21–24	Thesis Writing	Draft, edit, and finalize the thesis document.

Table 1: Proposed 24-week timeline for the thesis project.

References

- [1] Daniel Beßler et al. *Foundations of the Socio-physical Model of Activities (SOMA) for Autonomous Robotic Agents*. 2020. arXiv: 2011.11972 [cs.R0]. URL: <https://arxiv.org/abs/2011.11972>.
- [2] Manuel Borroto et al. *Question Answering with LLMs and Learning from Answer Sets*. 2025. arXiv: 2509.16590 [cs.AI]. URL: <https://arxiv.org/abs/2509.16590>.
- [3] Esra Erdem, Erdi Aker, and Volkan Patoglu. “Answer set programming for collaborative housekeeping robotics: representation, reasoning, and execution”. In: *Intell. Serv. Robot.* 5.4 (Nov. 2012), pp. 275–291. ISSN: 1861-2776. DOI: 10.1007/s11370-012-0119-x.
- [4] Federico Pennino et al. *From Reasoning to Code: GRPO Optimization for Underrepresented Languages*. 2025. arXiv: 2506.11027 [cs.LG]. URL: <https://arxiv.org/abs/2506.11027>.
- [5] Jan-Philipp Töberg, Svenja Kenneweg, and Philipp Cimiano. “RoboCSKBench: Benchmarking Embodied Commonsense Capabilities of Large Language Models”. In: *2025 22nd International Conference on Ubiquitous Robots (UR)*. College Station, Texas, USA, 2025, pp. 199–206. DOI: 10.1109/UR65550.2025.11078036.
- [6] Jan-Philipp Töberg et al. “Commonsense knowledge in cognitive robotics: a systematic literature review”. In: *Frontiers in Robotics and AI* 11 (2024), p. 1328934. DOI: 10.3389/frobt.2024.1328934.